

Assignment 3 — Bridge Pattern

Topic A — Drawing: Shape × Renderer

Course: ShP-2216 — Software Design Patterns (OP 6B06102)

Institution: Astana IT University — School of Software Engineering

Student: Adilzhan Kuandykov

Group: SE-2526

Language: Java (JDK 17)

Repository: <https://github.com/adilzhandev/design-pattern-bridge>

Commit: 70a798f85d88714003ff61110471175202beb35f

1. Two dimensions and why Bridge fits

My program draws shapes. A drawing has two parts that change for different reasons:

- **Shape dimension:** what is drawn — a Circle with radius 2 or a Square with side 3. This is the abstraction side (Shape).
- **Renderer dimension:** how it is drawn — as a vector path, as a pixel grid, or as ASCII art. This is the implementation side (Renderer).

If I used only inheritance, I would need one class for every pair: VectorCircle, RasterCircle, VectorSquare, RasterSquare, and then two more classes for ASCII. With n shapes and m renderers that is $n \times m$ classes.

Bridge splits the two dimensions into two hierarchies and connects them with one field: Shape keeps a reference of the interface type Renderer. Now I need only $n + m$ classes, and each side can grow on its own. The same object can also change its renderer while the program runs, which inheritance cannot do.

2. UML class diagram

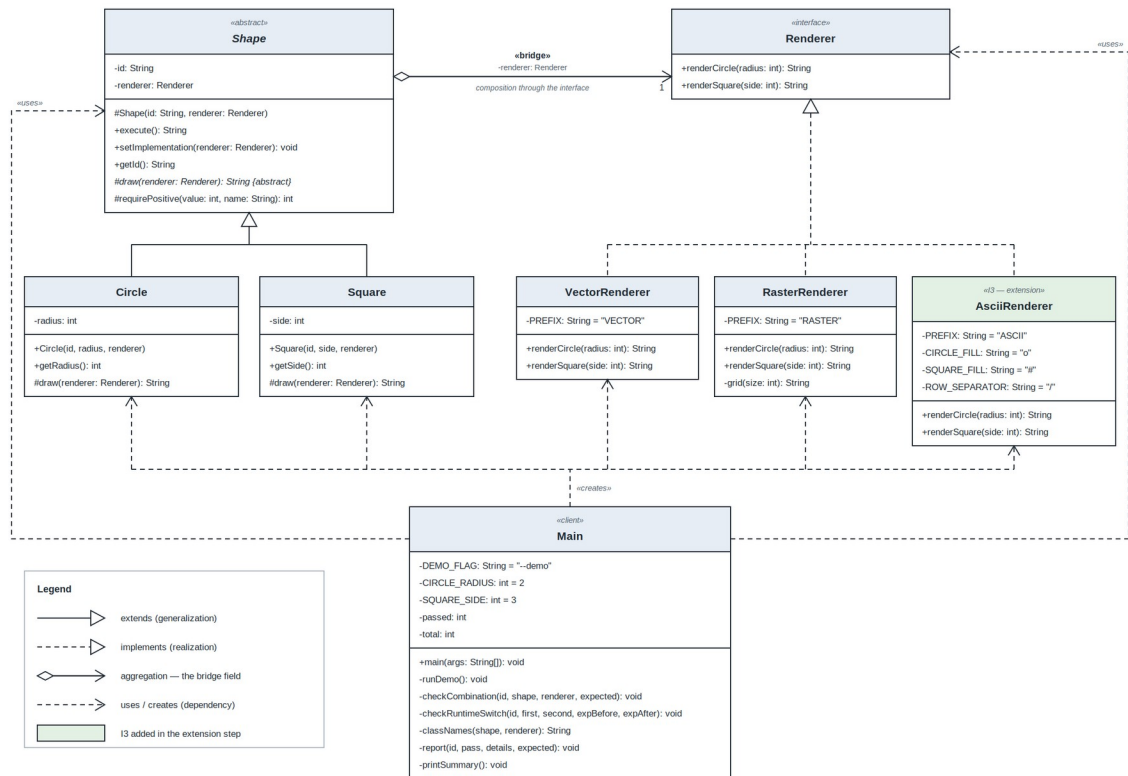


Figure 1 — Class diagram of the final code (also in docs/uml/class-diagram.svg)

Role	Class	Source path
Abstraction	Shape (abstract)	src/drawing/shape/Shape.java
A1 / A2	Circle / Square	src/drawing/shape/
Implementor	Renderer (interface)	src/drawing/renderer/Renderer.java
I1 / I2	VectorRenderer / RasterRenderer	src/drawing/renderer/
I3 (extension)	AsciiRenderer	src/drawing/renderer/AsciiRenderer.java
Client	Main	src/Main.java

The bridge is the field private `Renderer renderer` in `Shape`. It is given in the constructor and replaced with `setImplementation(Renderer)`. `execute()` calls the abstract `draw(renderer)`; `Circle` calls `renderer.renderCircle(radius)` and `Square` calls `renderer.renderSquare(side)`. `Main` creates the concrete objects and works with them through the `Shape` and `Renderer` types.

3. Clean Code excerpts

3.1 Separation of responsibilities

```
public String execute() {
    return draw(renderer);
}

@Override
protected String draw(Renderer renderer) {
    return renderer.renderCircle(radius);
}
```

Benefit: Shapes know their data, renderers know the output format, and Main only runs the checks. A shape never creates or casts a renderer, so a change in the output format cannot break shape code.

3.2 Meaningful names

```
private static final int CIRCLE_RADIUS = 2;
private static final int SQUARE_SIDE = 3;
public void setImplementation(Renderer renderer)
private void checkRuntimeSwitch(String checkId, Renderer first, Renderer second, ...)
```

Benefit: The names say what the value or method is for, so the demo reads like the task table and I do not need comments. The sample values 2 and 3 are written only one time.

3.3 Small, focused methods

```
@Override
public String renderSquare(int side) {
    return PREFIX + " square side=" + side + " pixels=" + grid(side);
}

private String grid(int size) {
    return size + "x" + size;
}
```

Benefit: Every method does one thing and is a few lines long. In Main, checkCombination() runs one check, report() prints and counts it, and printSummary() prints the total. Each one is easy to test and explain.

3.4 No duplicated workflow or formatting logic

```
checkCombination("T1", new Circle("C-1", CIRCLE_RADIUS, vector), vector, "VECTOR circle radius=2");
checkCombination("T2", new Circle("C-2", CIRCLE_RADIUS, raster), raster, "RASTER circle radius=2 pixels=4x4");

private void report(String checkId, boolean pass, String details, String expected) {
    total++;
    if (pass) {
        passed++;
    }
    System.out.println(checkId + " " + (pass ? "PASS" : "FAIL") + " | " + details);
    if (!pass) {
        System.out.println("    expected=" + expected);
    }
}
```

Benefit: T1–T4, T6 and T7 use the same checkCombination() method, and all seven checks print through one report() method. PASS is computed by comparing strings, never written by hand. Adding T6 and T7 took two lines. The check for a positive radius or side is also shared: requirePositive() is in Shape.

3.5 Encapsulation of state

```
public abstract class Shape {
    private final String id;
    private Renderer renderer;
    ...
    public void setImplementation(Renderer renderer) {
        this.renderer = Objects.requireNonNull(renderer, "renderer");
    }
}

public class Circle extends Shape {
    private final int radius;
```

Benefit: All fields are private. The ID and the radius are final, so the renderer switch in T5 cannot change them. The renderer can only be changed through `setImplementation()`, which refuses null. Subclasses get the renderer only as a parameter of `draw()`, not as a protected field.

4. Runtime switch (T5)

Main creates one Circle C-5 with VectorRenderer and saves the reference in original. It saves the ID and radius, calls `execute()`, then `setImplementation(raster)` and `execute()` again. `sameObject` is computed with `==`, so it proves that no new object was created. Only the renderer reference changes; the ID C-5 and radius 2 stay the same, and the result changes from VECTOR to RASTER.

5. Adding I3, trade-off and Bridge vs Adapter

Extension. I first finished the 2×2 version and the runtime switch and committed it as the base commit f792bfc. Then I added AsciiRenderer, which implements Renderer, and two checks in Main. `extension.diff (git diff f792bfc HEAD -- src)` shows only two files: the new AsciiRenderer.java (+24 lines) and Main.java (+6 lines). Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer did not change. This works because shapes depend only on the Renderer interface.

Trade-off. Bridge makes adding a renderer cheap, but adding a shape is not. Renderer has one method per shape (`renderCircle`, `renderSquare`). A new Triangle would need `renderTriangle()` in the interface, and then every renderer must change. Bridge also adds an extra layer: for a project with one renderer it would be more code than needed.

Bridge vs Adapter. Both patterns put an interface between two classes, but the intent is different. Adapter is used after the fact: it wraps an existing class with a wrong interface so a client can use it. Bridge is planned from the start: I designed Shape and Renderer together so that they can vary independently. In my project nothing is converted — VectorRenderer, RasterRenderer and AsciiRenderer were written to implement Renderer. If I had an old library class like LegacyPlotter.`plot(x, y, r)`, I would write an Adapter that implements Renderer and calls it; that adapter would then be one more implementor in my Bridge.

6. Demo output

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2 pixels=4x4
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3 pixels=3x3
T5 PASS | Circle + VectorRenderer -> RasterRenderer | sameObject=true | stateUnchanged=true
(id=C-5, radius=2)
    before=VECTOR circle radius=2 | after=RASTER circle radius=2 pixels=4x4
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2 art=(oooo)
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3 art=###/###/###
SUMMARY: 7/7 PASS
```

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

7. References

- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994 — Bridge (p. 151) and Adapter (p. 139).
- Shvets, A. Refactoring.Guru — Bridge. <https://refactoring.guru/design-patterns/bridge>
- Martin, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
- Oracle. Java SE 17 API Specification. <https://docs.oracle.com/en/java/javase/17/docs/api/>
- Bekenov, Y. Assignment 3 | Bridge Pattern — task description, ShP-2216, Astana IT University, 2026.